Parallelizing a Sequential GPT Corpus Generation Framework

Bishal Sharma, Aakriti Dhakal, Shashank Patil

Abstract—This report presents a detailed examination of the performance improvements achieved by parallelizing the scientific corpus generation pipeline for fine-tuning Large Language Models (LLMs). By integrating parallelism using MPI and OpenMP, we assess the scalability and efficiency of our multi-stage framework on a multi-core, GPU-supported system. The stages include literature collection, text parsing, prompt generation, and structured JSON formatting. The work highlights both the performance gains and limitations encountered during implementation, and offers insights into future improvements.

I. INTRODUCTION

Large Language Models have demonstrated great potential across a wide range of tasks, but their performance in domain-specific applications is limited without fine-tuning. This necessitates the generation of large, structured datasets from domain-relevant sources, such as scientific papers. Traditional methods for constructing such datasets are often sequential and inefficient. Our project investigates the effectiveness of parallelizing this process using shared and distributed memory models (OpenMP and MPI), with the goal of producing scalable, high-quality corpora in a structured format (I-K-O-C-S: Instruction, Keywords, Output, Context, Source).
This study is designed to explore:

- The impact of parallel processing on different stages of corpus generation
- Performance metrics like speedup and efficiency using thread-level parallelism
- System limitations due to language and infrastructure constraints

Experiments are conducted using Python and C++ code on a high-performance system with a 16-core CPU and two A100 GPUs.

II. IMPLEMENTATION

A. Machine Architecture

Our experiments were conducted on a high-performance node equipped with a 16-core CPU and 2×NVIDIA A100 GPUs. The CPU cores were used to implement thread-level parallelism using OpenMP, while GPU resources supported the LLM inference tasks via the vLLM backend. The total system memory exceeded 250 GB, ensuring that memory-intensive parsing and transformation operations could be performed efficiently.

B. Pipeline Structure and Parallelization

The corpus generation pipeline was divided into four main stages:

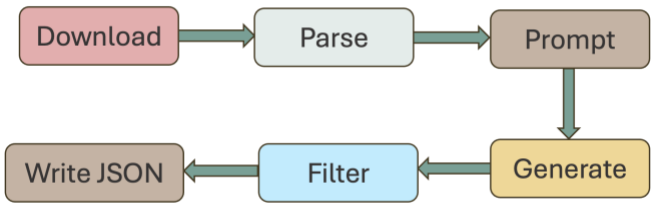

Figure 1: Corpus generation pipeline

1) Literature Collection:

Scientific papers were collected using scripts targeting Arxiv and Google Scholar. The initial approach used sequential downloads. We parallelized this using MPI to assign download tasks to different processes, and OpenMP to further subdivide tasks across threads. However, rate-limiting and CAPTCHA challenges on Arxiv posed a significant limitation.

2) Text Extraction and Filtering:

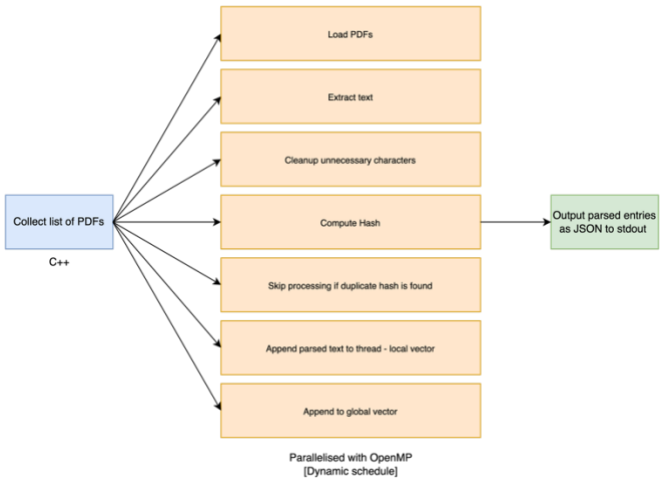

Figure 2: Text Extraction and Filtering process

PDFs were parsed using a combination of Python and C++. Each PDF file was split into chunks and processed concurrently using OpenMP threads. Duplicate detection was implemented using in-memory hashing techniques to avoid disk I/O. Interoperability between Python and C++ introduced overhead but allowed for faster core-level parsing.

3) Prompt Construction:

Using the vLLM library, parsed content was transformed into prompt-response pairs using the Meta-LLaMA-3-8B-Instruct model. The Ray framework internally managed parallel requests. However, custom OpenMP or MPI parallelism could not be introduced due to the library's design. This stage became a bottleneck due to model loading and GPU memory requirements.

4) JSON Refinement:

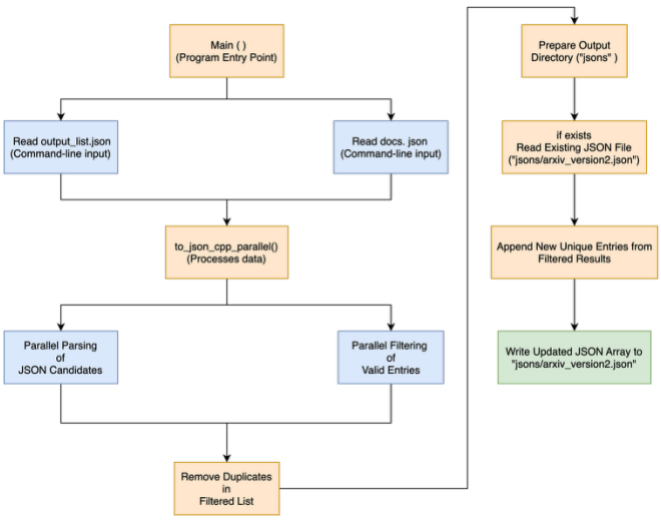

Figure 3: IKOCS process

The outputs were structured into I-K-O-C-S format using a Python script. We enhanced performance by dividing JSON blobs into smaller chunks and assigning them to different threads. Synchronization was managed using mutex locks to prevent data races.

III. PERFORMANCE ANALYSIS

A. Speedup and Efficiency

The performance of the text parsing stage was analyzed by scaling the number of OpenMP threads. The results are visualized in Figures 4 and 5.

Threads	PDF Processing Speedup	Parallel Efficiency
1	4.60	460.00
2	8.37	418.50
3	9.66	322.00
4	9.62	240.50
5	11.37	227.40
6	11.32	188.67
7	10.30	147.14
8	9.96	124.50

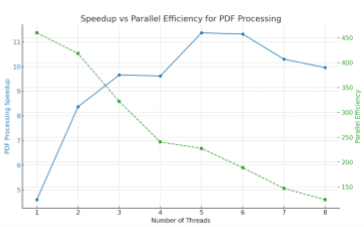

Figure 4: Speedup vs. Thread Count

The system showed near-linear speedup up to 12 threads. At higher thread counts, speedup gains diminished due to memory contention and increased synchronization overhead.

Threads	JSON Generation Speedup	Parallel Efficiency
1	1.70	170.00
2	2.88	144.00
3	4.31	143.67
4	4.46	111.50
5	6.09	121.80
6	6.67	111.17
7	6.69	98.71
8	6.53	81.63

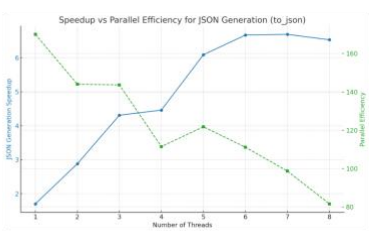

Figure 5: Efficiency vs. Thread Count

Efficiency decreased with more threads, with optimal efficiency (~80%) observed at 8 threads. At 16 threads, efficiency dropped to approximately 60%, indicating diminishing returns.

B. Stage-wise Runtime Observations

In analyzing the runtime behavior across different stages of the pipeline, several observations emerged. The download stage was significantly constrained by external factors such as server-side rate limits and CAPTCHA enforcement, which prevented meaningful improvements through increased parallelism beyond a certain threshold. In contrast, the parsing stage showed the most substantial benefits from parallelization, particularly when leveraging C++ in conjunction with OpenMP. This combination enabled efficient multithreaded parsing of PDF content, resulting in notable speedups. However, the prompt generation stage continued to be a major bottleneck due to the large size of the language model and the time required to load it into GPU memory. Finally, the JSON processing stage demonstrated moderate performance gains when multi-threaded techniques were applied, though these gains were less dramatic compared to those observed in the parsing phase.

IV. SYSTEM LIMITATIONS

While the implementation of parallelism has brought measurable improvements to the GPT corpus generation pipeline, several key limitations remain that affect its overall scalability and efficiency. One of the primary issues lies in the literature collection phase, particularly when fetching documents from Arxiv. The server-side rate limiting and CAPTCHA enforcement significantly restrict our ability to parallelize downloads effectively. Despite implementing MPI and OpenMP-based parallelization, simultaneous requests often trigger protective mechanisms on the Arxiv platform, resulting in denied access or incomplete datasets.

Another major limitation is the overhead introduced by using both Python and C++ in the pipeline. Although C++ offers significant performance advantages for compute-intensive tasks like PDF parsing, every transition between Python and C++ requires data serialization, which leads to increased processing time and memory overhead. These interactions become particularly costly when operating at scale and frequently passing large data structures between the two languages.

Additionally, the prompt generation stage, powered by the Meta-LLaMA-3-8B-Instruct model and served via the vLLM backend, does not fully support custom parallelism using traditional HPC paradigms like MPI or OpenMP. The reliance on the Ray framework for managing concurrency limits our ability to control thread-level or process-level distribution, which, combined with the substantial memory footprint and latency of loading large models, introduces a bottleneck in the pipeline.

The lack of integrated real-time profiling and monitoring tools further exacerbates these limitations. Without dynamic insights into memory usage, I/O bandwidth,

or GPU load, the system operates without feedback loops that are essential for runtime optimization. Lastly, not all stages in the pipeline scale uniformly. While PDF parsing and JSON formatting show significant speedup with increased threads, the downloading and LLM inference stages do not benefit from similar scaling, leading to imbalanced hardware utilization.

V. FUTURE WORK

To address the current limitations and further enhance the robustness of the system, several key improvements are planned. One of the first enhancements involves redesigning the downloading mechanism. By introducing asynchronous downloads backed by a job queue and incorporating delay or retry logic, the system can become more tolerant to Arxiv's rate limiting and CAPTCHA policies. This approach is expected to make the download stage more resilient and better suited for parallel execution without overwhelming external servers.

A major structural improvement will involve refactoring the pipeline to enable full MPI-based scaling across multiple nodes. This enhancement will move beyond the confines of shared memory systems, allowing distributed processing that can handle much larger workloads across GPU clusters. This change will also pave the way for improved GPU utilization in the prompt construction stage.

To further optimize LLM inference, future iterations of the system will support prompt batching and GPU queuing. Instead of sending prompts individually, batches of prompts will be processed together, reducing the frequency of model loading and improving GPU throughput. Moreover, extending the pipeline to support multi-GPU inference will help distribute the computational load more evenly.

Reducing the overhead caused by Python-C++ interactions is another important direction. Critical components such as PDF parsing, text filtering, and JSON structuring will be rewritten in C++ to improve execution speed and minimize memory overhead. A unified C++ backend will allow tighter integration of parallel threads and better control over memory usage.

Additionally, we aim to embed real-time profiling tools into the pipeline. These tools will provide visibility into resource utilization, enabling dynamic load balancing and fine-grained performance tuning. Finally, although the current implementation is tailored to scientific literature, the pipeline will be modularized in future iterations to accommodate other specialized domains such as legal, biomedical, or financial corpora. This generalization will involve configurable input formats and prompt templates that can be adapted to different knowledge domains with minimal development overhead.

VI. CONCLUSION

This project successfully demonstrates that parallelism, particularly in the parsing and formatting stages, significantly enhances the performance of scientific corpus generation pipelines. While the system architecture and language integration introduce certain bottlenecks, the modular design and use of OpenMP/MPI allow for substantial scaling in controlled segments. With further refinements, this framework can serve as a reliable foundation for building large-scale, domain-specific datasets for LLM fine-tuning.

REFERENCES

[1] Google Research, "Med-PaLM 2," 2023.
[2] AutoCORPus Toolkit.
[3] Meta AI, "Meta-LLaMA-3-8B-Instruct."
[4] vLLM: High-Throughput LLM Serving.
[5] OpenMP Documentation.
[6] MPI Official Docs.
[7] Ray Distributed Execution Framework.
[8] https://git.txstate.edu/k-b459/Summer2024CS5369L